Kilian Hinrichs
Angeline Usanayo

Aufgabe 1

Conceptual Integrity

- Unterschiedliche variable Benennungen bei *number1* und *num2*
- Unterschiedliche Formate in der Message mit ffirstünd "2nd"
- bei Subtraktion wird das Ergebnis erst lokal in *res* gespeichert und danach auf *result* geschrieben, während das bei den anderen Cases direkt passiert

Information Hiding

- Alle Variablen public statt private

Coupling

- Nur mit der Klasse *Scanner* vorhanden
- schlechtes Coupling, weil die Klasse hier zwangsläufig an *Scanner* gebunden ist und die Logik nicht in anderen Kontexten verwendet werden kann

Cohesion

- Alle Funktionen sind in *start()* implementiert und könnten auf mehrere Methoden aufgeteilt werden

Aufgabe 2

Code ist angehängt.

- Die Rechenlogik, der Input und der Oputput sind in unterschiedliche Klassen verlegt
  - damit arbeitet jede Klasse nach Single Responsibility Principle
- dadurch können auch einzelne Klassen ausgetauscht werden, bspw. um den Output in einer GUI zu haben

Aufgabe 3

a)

Aufgabe 4

- A2
- B4
- C1
- D5
- E3

Aufgabe 5

Aufgabe 6

- Dependency Inversion Principle ermöglicht, dass die Komponenten nicht mehr direkt voneinander abhängig sind, sondern ausgetauscht werden können
- Mock Objekte können Komponenten Simulieren, dass sie leichter getestet werden können, z.B. wenn die echte Abhängigkeit Ressourcen benutzt, die das Testen erschweren
- Hauptvorteil im Austausch der Komponenten

Aufgabe 7

- Dry: Don't repeat yourself ⇒ Name selbsterklärend
- Die Constructor können sich gegenseitig aufrufen, um die Dopplung in der Variable Setzung zu meiden
- Die bereits genutzten Parameter werden dann an den kürzeren Constructor übergeben und die nach unten hin neu eingeführten Parameter werden weiterhin selbst gesetzt